Core Services & HTTP Layer

<details>
<summary>Relevant source files</summary>

- `Backend/STUDER/src/main/resources/application-dev.properties`
- `Backend/STUDER/src/main/resources/application-prod.properties`
- `Frontend/STUDER/public/favicon.ico`
- `Frontend/STUDER/src/app/app.component.html`
- `Frontend/STUDER/src/app/app.component.ts`
- `Frontend/STUDER/src/app/app.config.ts`
- `Frontend/STUDER/src/app/config/api.config.ts`
- `Frontend/STUDER/src/app/core/auth/auth-api.service.ts`
- `Frontend/STUDER/src/app/core/auth/auth-state.service.ts`
- `Frontend/STUDER/src/app/core/http/case-converter.interceptor.ts`
</details>

The Core Services and HTTP Layer form the backbone of the STUDER Angular application, managing communication with the backend, authentication state, and data transformation. This layer ensures that the frontend maintains a consistent security posture and interacts with the REST API using standardized naming conventions regardless of backend preferences.

HTTP Interceptor Pipeline

The application employs a chain of four specialized interceptors configured in **`appConfig`** `Frontend/STUDER/src/app/app.config.ts 15` These interceptors process every outgoing request and incoming response to handle cross-cutting concerns like security, internationalization, and data formatting.

1. Case Converter Interceptor

The **`caseConverterInterceptor`** bridges the naming convention gap between the frontend (camelCase) and the backend (snake_case).

- **Outgoing Requests**: Recursively transforms the request body from camelCase to snake_case using `toSnakeCase`
  `Frontend/STUDER/src/app/core/http/case-converter.interceptor.ts` `21-32`
- **Incoming Responses**: Recursively transforms the response body from snake_case to camelCase using `toCamelCase`
  `Frontend/STUDER/src/app/core/http/case-converter.interceptor.ts` `8-19`
- **Exclusions**: Requests for assets or those containing `FormData` (e.g., file uploads) are ignored to prevent corruption of binary data
  `Frontend/STUDER/src/app/core/http/case-converter.interceptor.ts` `39-45`

2. Auth Interceptor

The `authInterceptor` is responsible for injecting the `Authorization: Bearer <token>` header into outgoing requests. It retrieves the current access token from `TokenStorageService`.

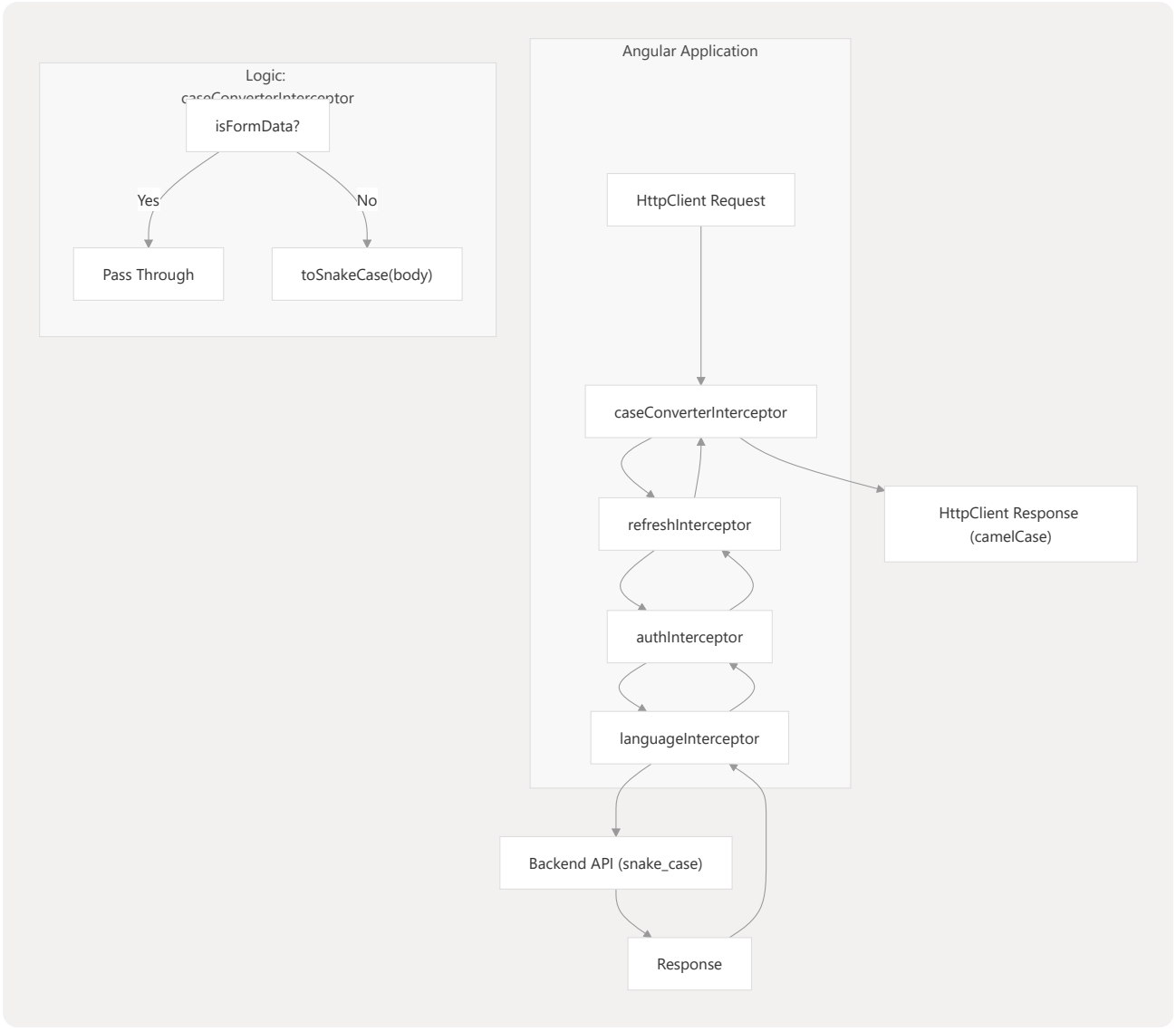
3. Refresh Interceptor

The `refreshInterceptor` handles token rotation. If a request fails with a `401 Unauthorized` error, this interceptor attempts to call the refresh endpoint to obtain a new access token before retrying the original request.

4. Language Interceptor

The `languageInterceptor` injects the `Accept-Language` header into every request, ensuring the backend returns localized error messages and content based on the user's selected language.

HTTP Data Flow Diagram



Sources:

- `Frontend/STUDER/src/app/core/http/case-converter.interceptor.ts` `1-61`
- `Frontend/STUDER/src/app/app.config.ts` `11-18`

Authentication & State Management

Authentication is managed through a tripartite service architecture that separates API communication, local storage, and reactive state.

AuthApiService

A stateless service that performs raw HTTP calls to the authentication and user endpoints defined in `API_CONFIG`.

- `me()`: Fetches the current user profile
  `Frontend/STUDER/src/app/core/auth/auth-api.service.ts` `17-19`
- `login(data)`: Posts credentials to `/auth/login`
  `Frontend/STUDER/src/app/core/auth/auth-api.service.ts` `25-27`
- `refresh()`: Requests a new JWT via `/auth/refresh`
  `Frontend/STUDER/src/app/core/auth/auth-api.service.ts` `29-31`

TokenStorageService

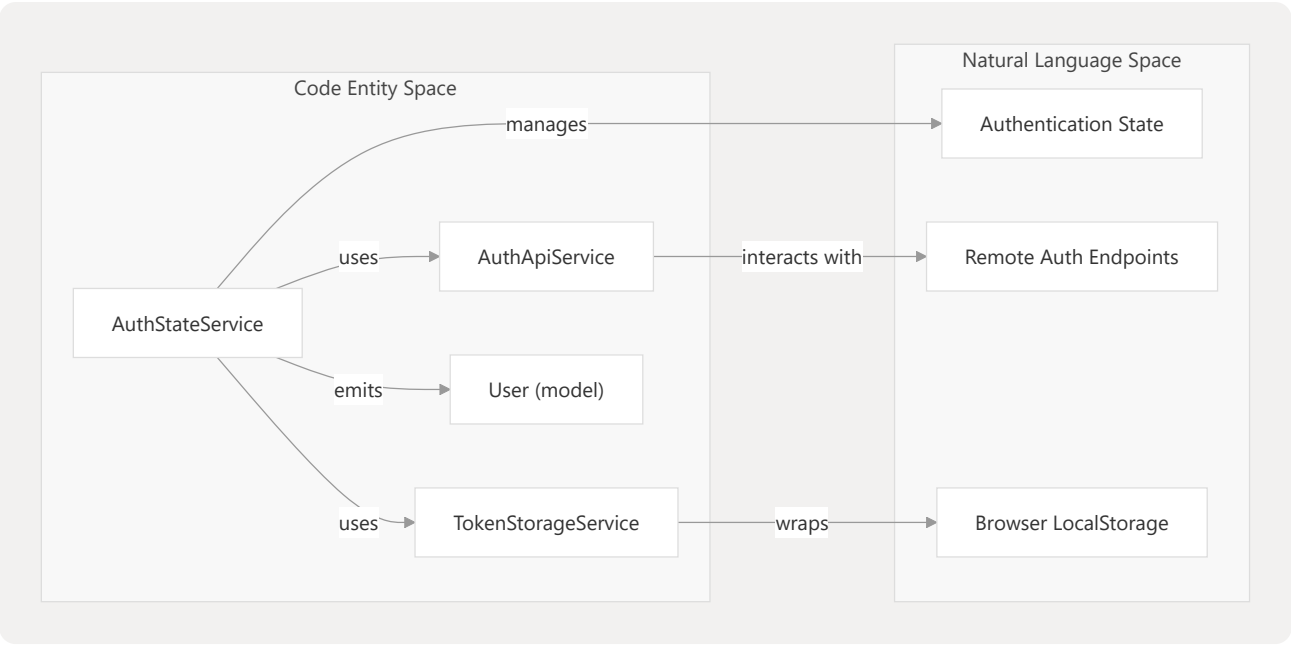
Handles the persistence of the JWT access token in the browser's storage. It provides methods like `setAccessToken`, `hasValidToken`, and `clear`.

AuthStateService

The central reactive hub for authentication. It uses a `BehaviorSubject` to track the current `User` object and exposes observables for components to react to login/logout events.

- `user$`: Observable of the current user
  `Frontend/STUDER/src/app/core/auth/auth-state.service.ts` `14`
- `isAuthenticated$`: Derived boolean observable
  `Frontend/STUDER/src/app/core/auth/auth-state.service.ts` `15`
- `login()`: Executes the login flow: calls `authApi.login`, stores the token, and then fetches the user profile via `me()` `Frontend/STUDER/src/app/core/auth/auth-state.service.ts` `42-53`

Auth State Entity Mapping



Sources:

- `Frontend/STUDER/src/app/core/auth/auth-api.service.ts` `1-38`
- `Frontend/STUDER/src/app/core/auth/auth-state.service.ts` `1-72`

API Configuration

The application centralizes its endpoint definitions in `API_CONFIG`. This ensures that changes to the backend URL or base paths only need to be updated in one location.

Constant	Value	Description
`baseUrl`	`http://localhost:1583/api/v1`	Base URL through Nginx proxy `Frontend/STUDER/src/app/config/api.config.ts` `2`
`auth`	`/auth`	Authentication endpoints (login, refresh, logout) `Frontend/STUDER/src/app/config/api.config.ts` `3`
`users`	`/users`	User profile and registration `Frontend/STUDER/src/app/config/api.config.ts` `4`

Constant	Value	Description
`discussions`	`/discussions`	Discussion threads and messages `Frontend/STUDER/src/app/config/api.config.ts 6`
`friends`	`/friends`	Social graph and following logic `Frontend/STUDER/src/app/config/api.config.ts 7`

Sources:

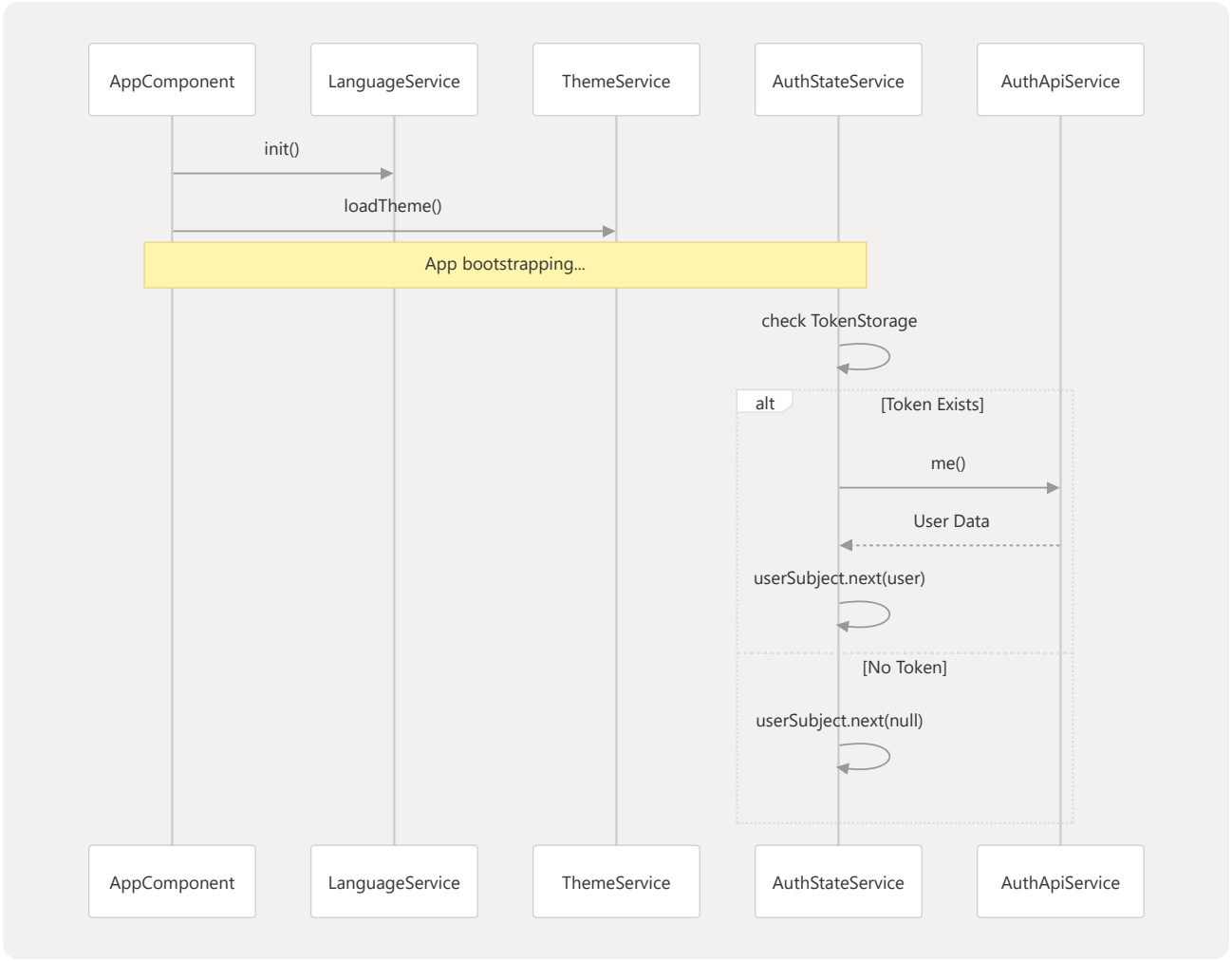
- `Frontend/STUDER/src/app/config/api.config.ts 1-9`
- `Backend/STUDER/src/main/resources/application-prod.properties 19-22`

Initialization Flow

During application bootstrap, `AppComponent` initializes the core services to restore the user session and visual preferences.

1. **Language & Theme**: `AppComponent.ngOnInit` calls `LanguageService.init()` and `ThemeService.loadTheme()` `Frontend/STUDER/src/app/app.component.ts 21-24`
2. **Auth Restoration**: `AuthStateService.init()` is called (typically by a guard or component). It checks for a valid token; if found, it calls `authApi.me()` to restore the reactive user state `Frontend/STUDER/src/app/core/auth/auth-state.service.ts 22-36`

System Startup Sequence



Sources:

- `Frontend/STUDER/src/app/app.component.ts` `13-25`
- `Frontend/STUDER/src/app/core/auth/auth-state.service.ts` `22-36`